Paper Representation

m11415067 周珥豪

Data Cache Prefetching Using a Global History Buffer

Kyle J. Nesbit and James E. Smith
Department of Electrical and Computer Engineering
University of Wisconsin - Madison
{nesbit, jes}@ece.wisc.edu

10th International Symposium on High Performance Computer Architecture (HPCA'04)

Outline

1. Introduction
2. Table-Based Prefetching
3. Global History Buffer Prefetching
4. Performance Evaluation
5. Conclusion

Introduction

- Background
  - Processor speed keeps increasing, but main memory hasn't kept up.
    → Memory access latency (in cycles) becomes relatively longer.

- Why data cache prefetching is needed
  - Multi-level caches help hide latency, but cache misses still happen.
    → Data prefetching is needed to further reduce memory stall time.

Introduction

- Prefetching method
  1. Sequential prefetching
     - The simplest way to prefetch
  2. table-based prefetching
     - Record history information related to access data

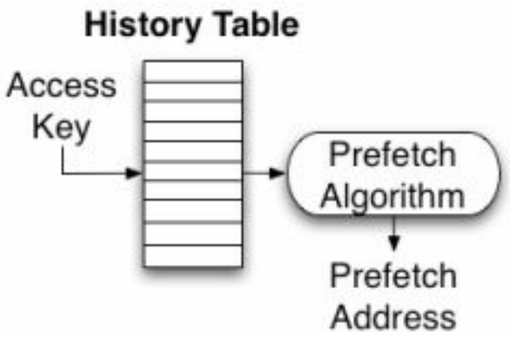


Figure 1a: Basic Prefetch Table

Cons:

1. stale data
2. Low space utilization
3. High error rate

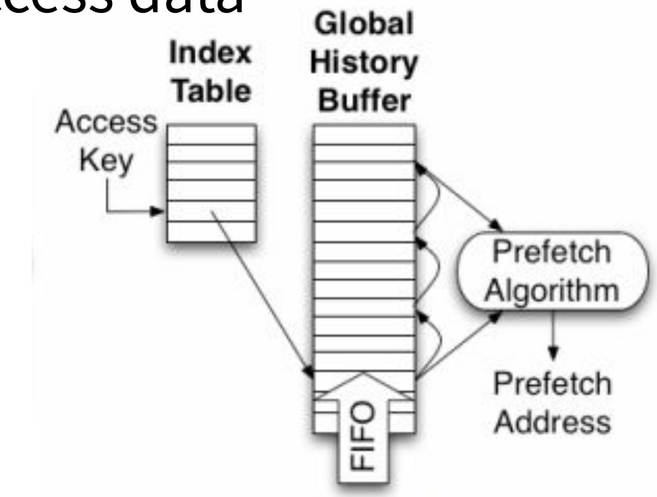


Figure 1b: Global History Buffer Prefetch Structure

(Linklist + hash table)

Table-based Prefetching

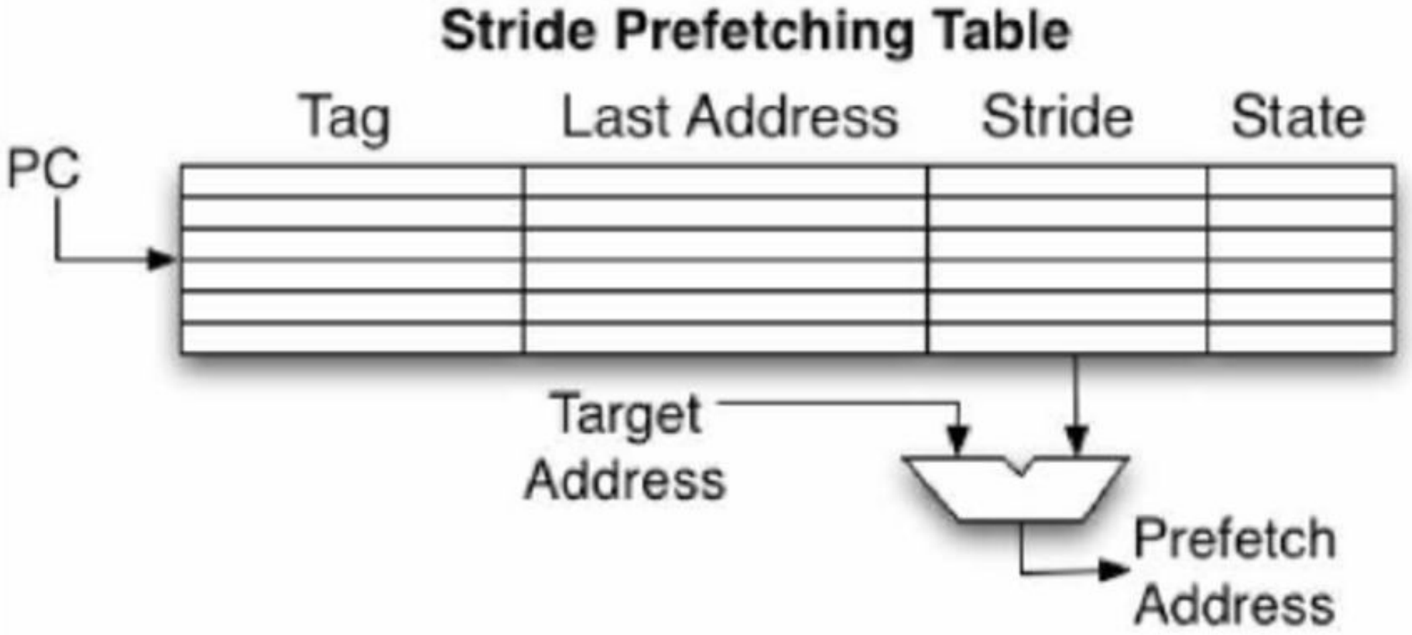
- Def: Use table to store past access history, and use PC or miss address of the load instruction as the index key to predict the next address to be fetched from the table.

- Three methods
  1. Stride Prefetching
  2. Markov Prefetching
  3. Distance Prefetching

1.Stride Predfetching

- Def:
  - Store past access history in a table.
  - Use PC or miss address as the index to predict the next access.
- Operation: <u>If stride is confirmed to be stable, it</u> prefetches:

  a + s , a + 2s , ... , a + ds (a is address, d is prefetch degree, s is stride)



2.Markov Prefetching

- Def: Use address correlation to predict the next address that will miss.
- Operation: if miss A → next likely B or C.
- Markov graph: Node is miss address, arc is probabiity.

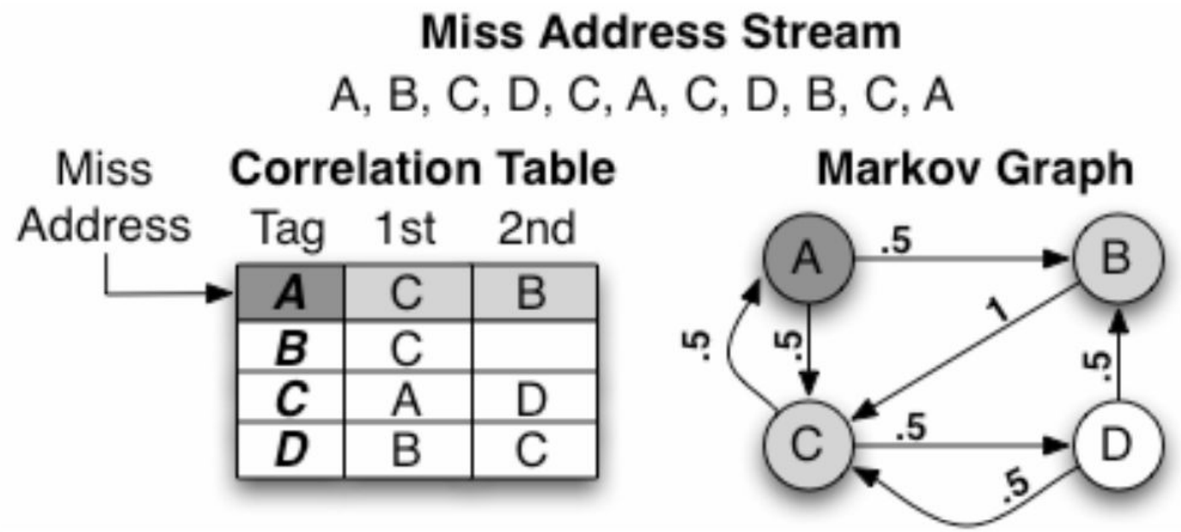


Figure 3: Markov Prefetching

3.Distance Prefetching

- Def: Based on the "distance between two consecutive miss addresses (delta)".
- Operation: When the same delta pattern is detected, these deltas are added to the current miss address to calculate the prefetch target.
- Pros: More expressive, can handle non-fixed stride

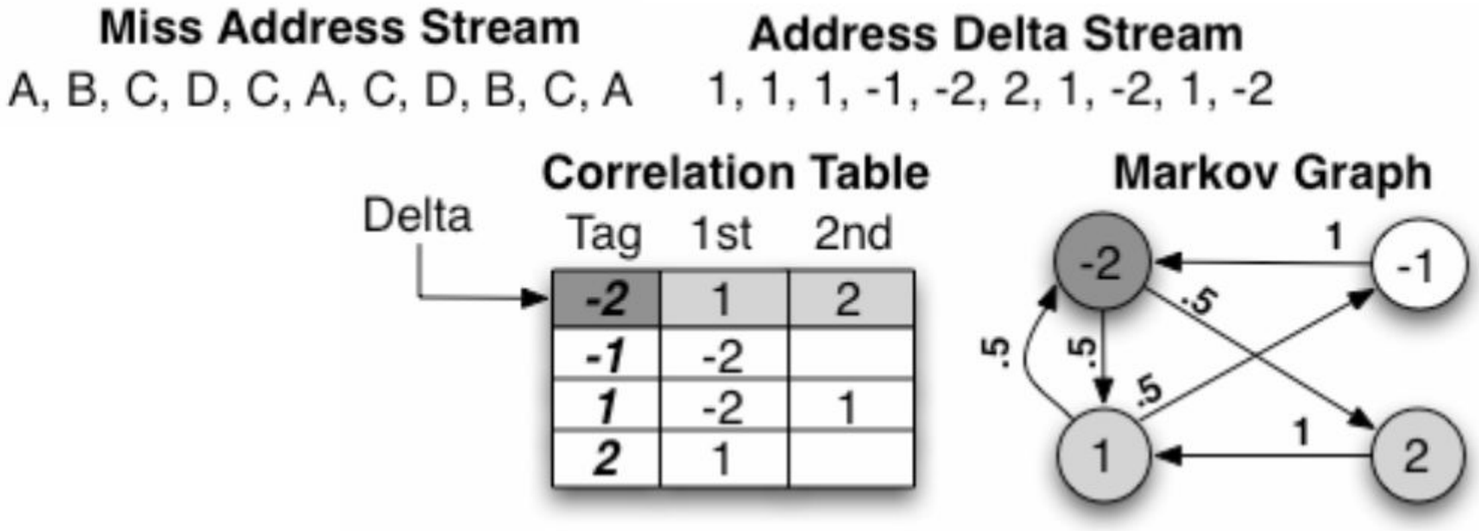


Figure 4: Distance Prefetching.

Global History Buffer Prefetching

- Problems with traditional table-based prefetching:

1. Stale Table data reduce the accuracy of prefetching.
2. Index conflict
3. Limited history per entry

- GHB's Core Concepts :

  Two-Level Structure:

  1. Index Table

     - Points to recent entries in GHB

  2. Global History Buffer → a FIFO circular buffer

     - stores miss history
     - linked by pointers to reconstruct access sequence and avoid stale data

GHB implementation

- Operation

  Every time a L2 Cache miss occurs:

  1. Update the GHB using a FIFO policy.
  2. Establish a link
  3. Update Index Table
  4. Increment the GHB head pointer.

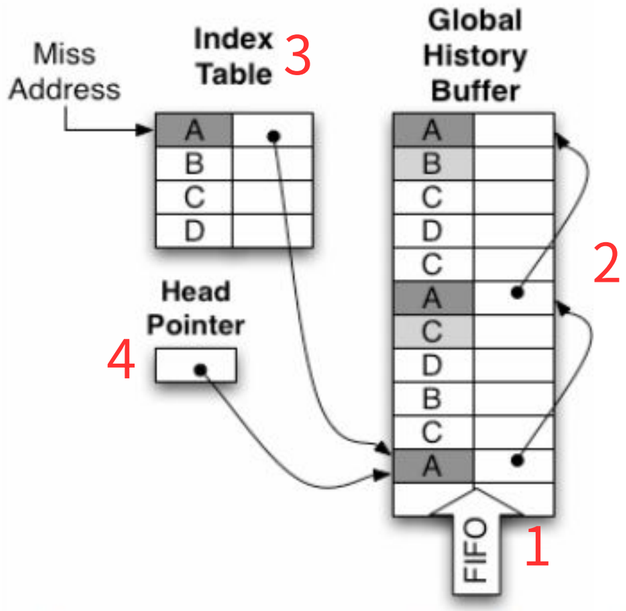


Figure 5: GHB Global / Address Correlation

- Error and detection mechanism

  - GHB is a circular buffer, so old entries may be overwritten.

    → sol: Expand pointer width, if distance > GHB size → pointer invalid.

Global History Buffer Prefetching

- Classification of prefetching methods
  each denoted as a pair X/Y
  1. X: Program Counter & Global
  2. Y: Constant Stride, Delta Correlation, and Address Correlation

📘 GHB Prefetching Method Taxonomy

X (Localization)	Y (Pattern Detection)
PC	CS / DC / AC
G	CS / DC / AC

Ex: Markov Prefetching(G/AC)

- Core Concept & Operation
  1. With GHB, the misses are linked as a linked list
  2. GHB automatically maintains recent history
  3. Each next node = miss that followed it before.
  4. Prefetch candidates are the next misses connected in these linked lists.

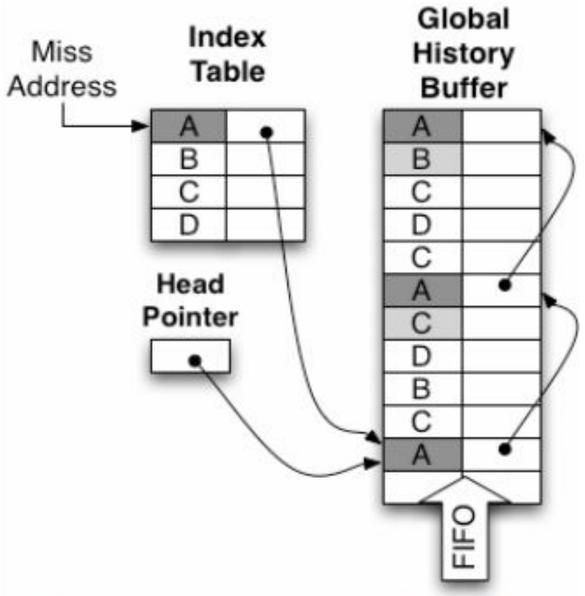


Figure 5: GHB Global / Address Correlation

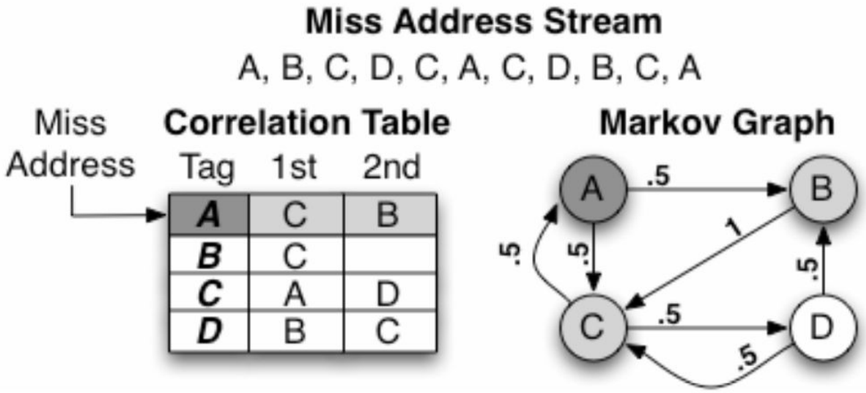


Figure 3: Markov Prefetching

Same as traditional Markov Prefetching

Ex: Stride Prefetching(PC/CS)

- Core Concept & Operation
  1. Traditional stride prefetching only tracks the last address & stride.
  2. With GHB, it records a sequence of miss addresses for each PC.
  3. This allows more accurate stride detection and avoids errors from short repeating patterns.

Generalized Correlation Prefetching

- Markov-based correlation prefetching can be extended to
  1. Width Prefetching
     → Prefetch neighboring nodes. (stable patterns)
  2. Depth Prefetching
     → Prefetch along the most likely path. (Increase lookahead)
  3. Hybrid

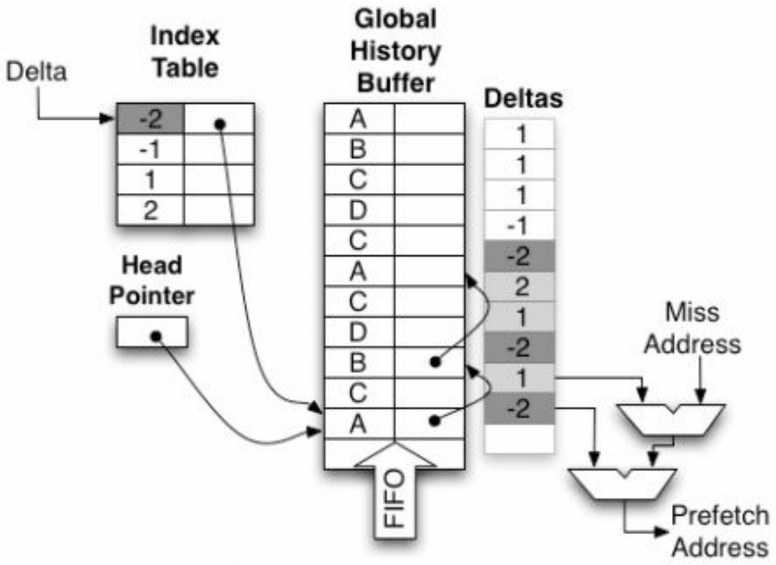


Figure 6: GHB Global / Delta Correlation

Depth Prefetching

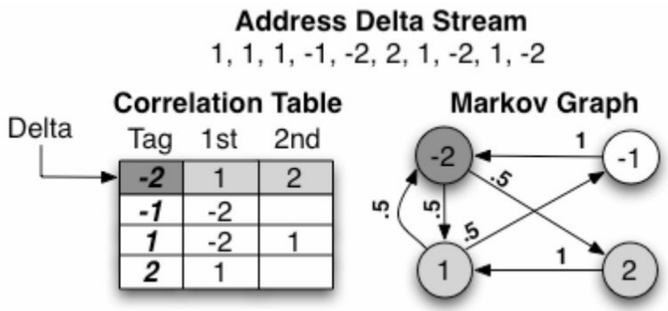


Figure 4: Distance Prefetching.

Local Delta Correlation

- Traditional PC/CS only compares the last stride
  → may mispredict (e.g. repeating 1s).
- Improvement methods(PC/DC) is that GHB records the actual sequence of miss addresses, allowing patterns between pairs of consecutive deltas to be observed
  → So it use Delta Pair(D1, D2) to be a correlation key.

Addresses	0	1	2	64	65	66	128	129
Deltas		*1*	*1*	*62*	*1*	*1*	*62*	*1*

Table 1: Example Address Stream Correlations

Local Correlation Key (most recent delta pair)	**Prefetch Prediction (4 subsequent deltas)**			
(1, 1)	*62*	*1*	*1*	*62*
(1, 62)	*1*	*1*	*62*	*1*
(62, 1)	*1*	*62*	*1*	*1*

Simulation Methodology

- Subset of SPEC CPU2000 (exclude sixtrack due to syscall issue).
- Simplescalar 3.0
- Index Table & GHB access latency = 1 cycle
- Evaluation Focus

  - Table G/DC & GHB G/DC

  - Table PC/CS & GHB PC/CS

  * Excluding Markov Prefetching (G/AC).

    → Its performance is generally lower than G/DC, and it requires excessive storage.

Simulation Methodology

Table 2: SPEC FP Subset with Ideal L2 IPC Improvement.

Benchmark	IPC (%)
ammp	815%
art	238%
wupwise	204%
swim	136%
lucas	135%
mgrid	56%
applu	36%
galgel	30%
apsi	20%

Table 3: SPEC INT Subset with Ideal L2 IPC Improvement.

Benchmark	IPC (%)
Mcf	319%
twolf	104%
vpr	78%
parser	40%
gap	22%
bzip2	19%

Table 4: Simulator Configuration

Issue Width	4 instructions
Load Store Queue	64 entries
RUU Size	128 entries
Level 1 D-Cache	16KB, 2-way set associative
Level 1 I-Cache	16KB, 2-way set associative
Level 2 Cache	512KB, 2-way set associative
Memory Latency	140 Cycles

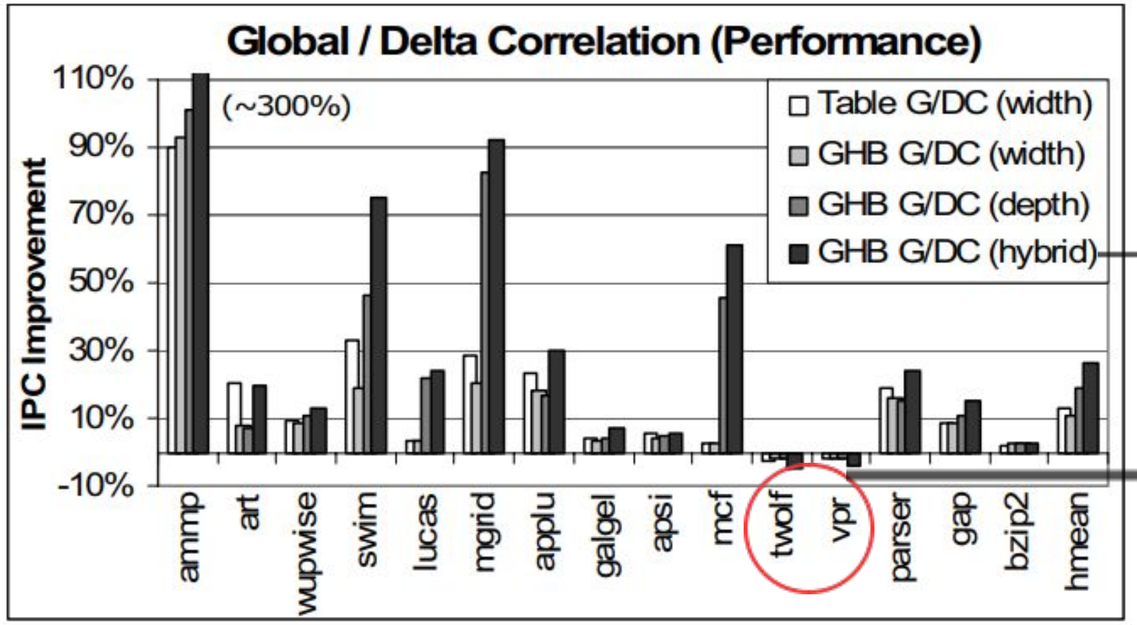
SimpleScalar 3.0

GHB Prefetch Performance

- Three evaluation indicators
  1. IPC Improvement (per benchmark)
  2. IPC Improvement (vs. Prefetch Degree)
  3. Increase in Memory Traffic
- Evaluation Targets
  1. Global / Delta Correlation
  2. PC Local Prefetching
- Prefetch Degree = 4

Global / Delta Correlation

- GHB G/DC hybrid improves overall IPC by 13%.



width*depth=16

Excessive prefetching

Figure 7: Global / Delta Correlation (prefetch degree four) IPC Improvement per benchmark.

Global / Delta Correlation

- It is proved that Table G/DC has stale data, which causes the prefetch accuracy to decrease.

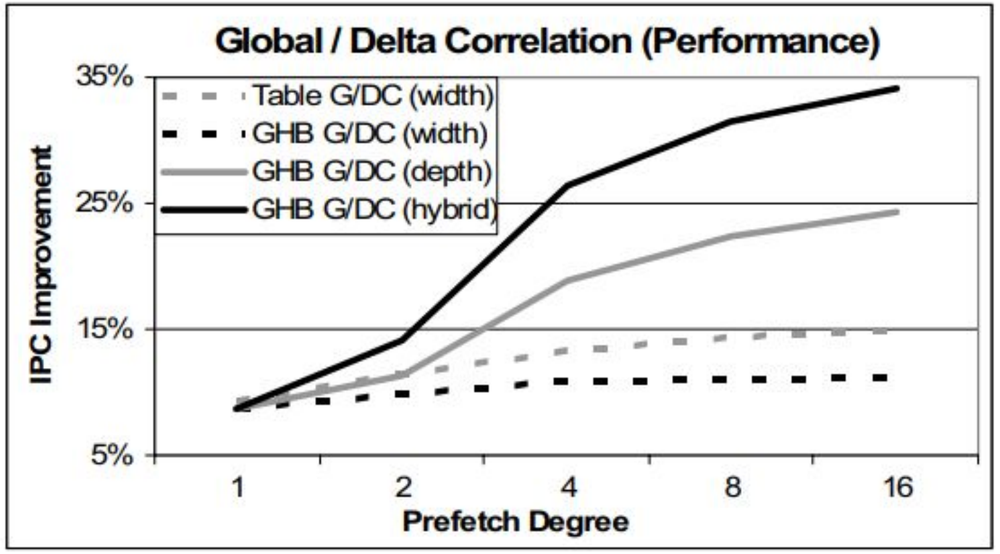


Figure 8: Global / Delta Correlation IPC Improvement for prefetch degrees 1 through 16.

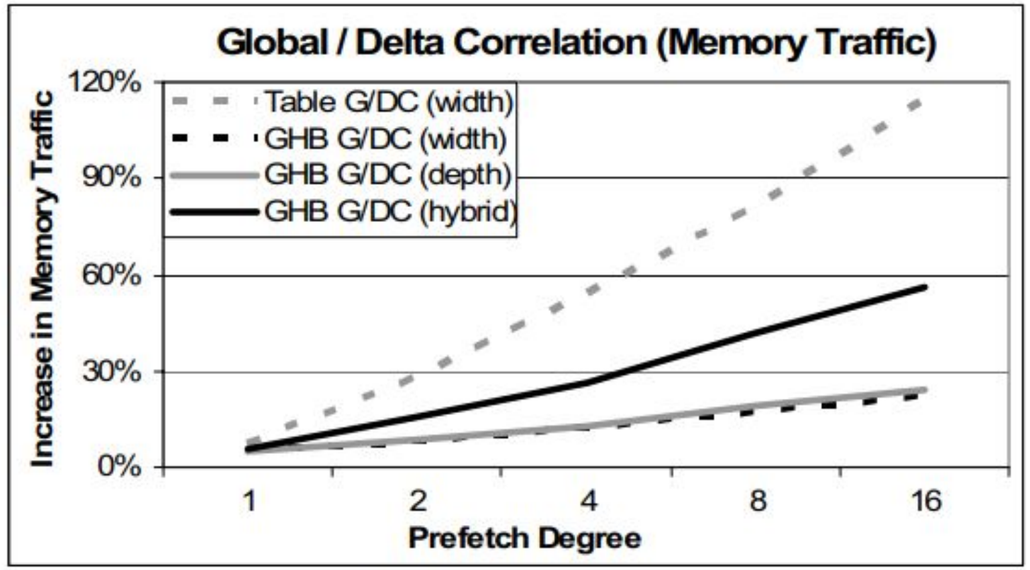


Figure 9: Global / Delta Correlation Increase in Memory Traffic for prefetch degrees 1 through 16.

Global / Delta Correlation

- Stale Table Data Behavioral Analysis

  (age is the number of cycles since the last access)
- Conclusion: Older table data is more likely to lead to incorrect prefetching.

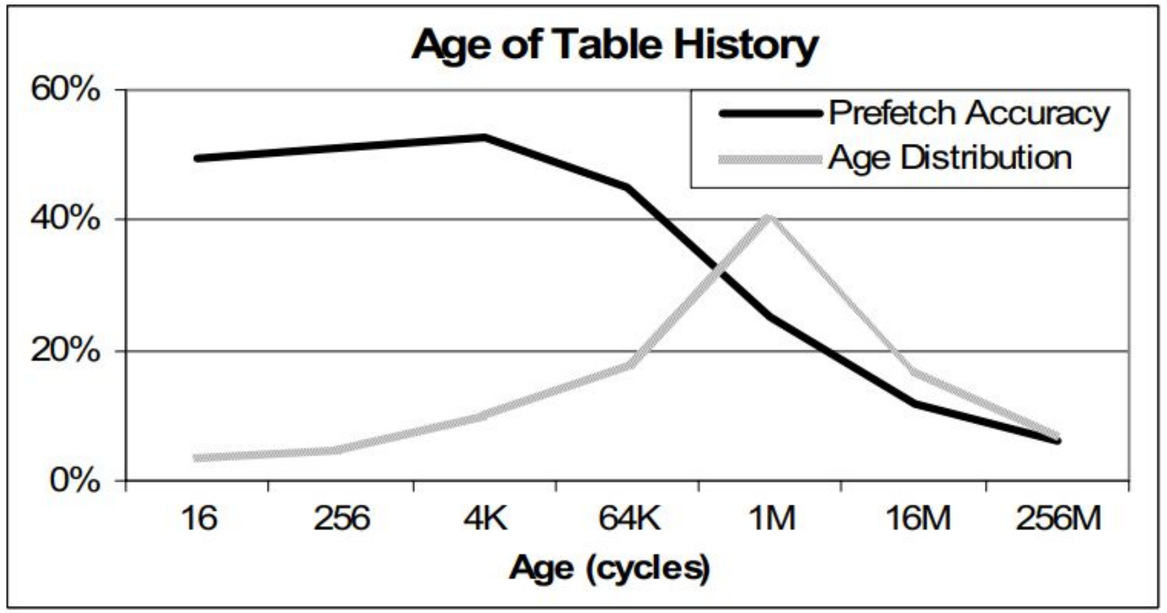


Figure 10: Age distribution of Table History that generates a prefetch and prefetch accuracy per age group.

PC Local Prefetching

- GHB PC/DC and GHB G/DC (hybrid) have similar overall performance

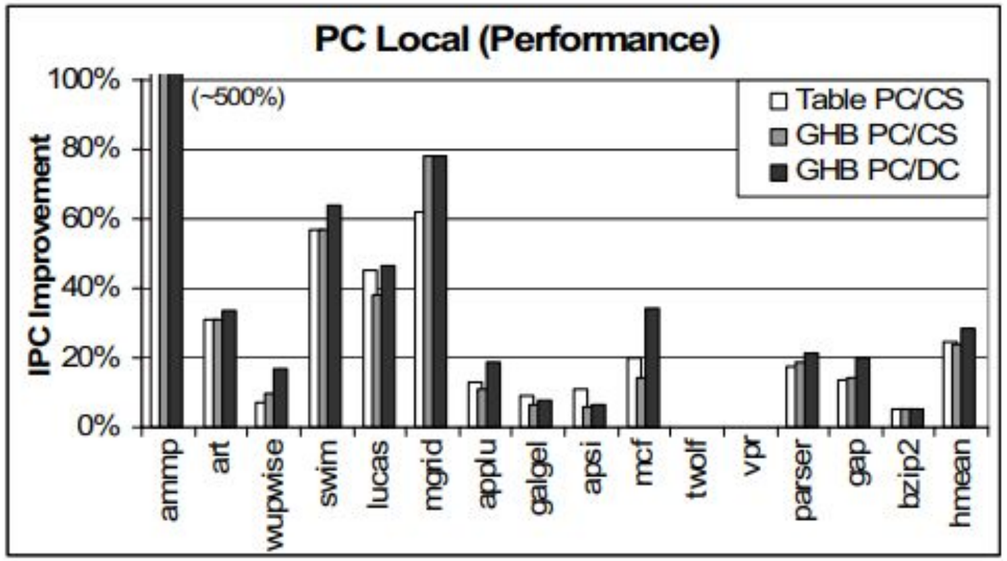


Figure 11: PC Local methods (PC/CS and PC/DC) (prefetch degree four) IPC Improvement per benchmark.

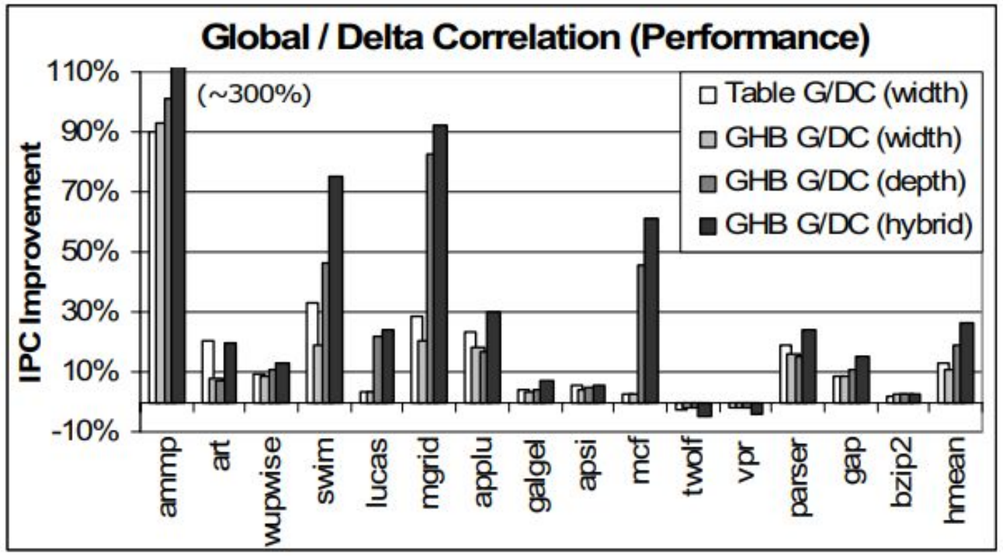


Figure 7: Global / Delta Correlation (prefetch degree four) IPC Improvement per benchmark.

PC Local Prefetching

- GHB PC/DC combines the advantages of stride and delta prefetching

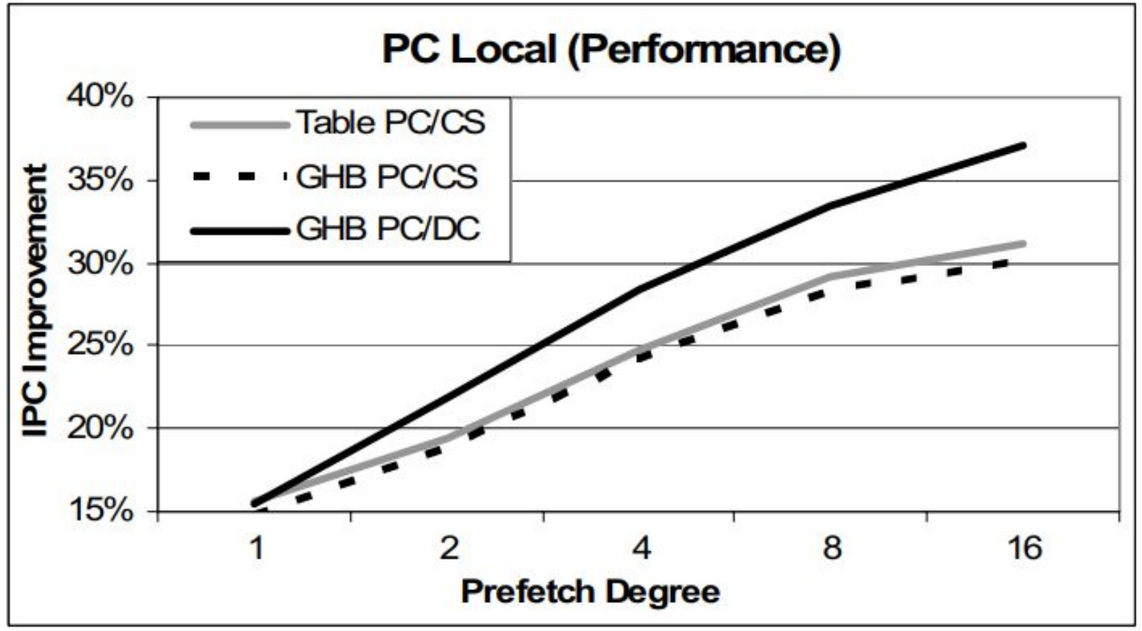


Figure 12: PC Local methods (PC/CS and PC/DC) IPC Improvement for prefetch degrees 1 through 16.

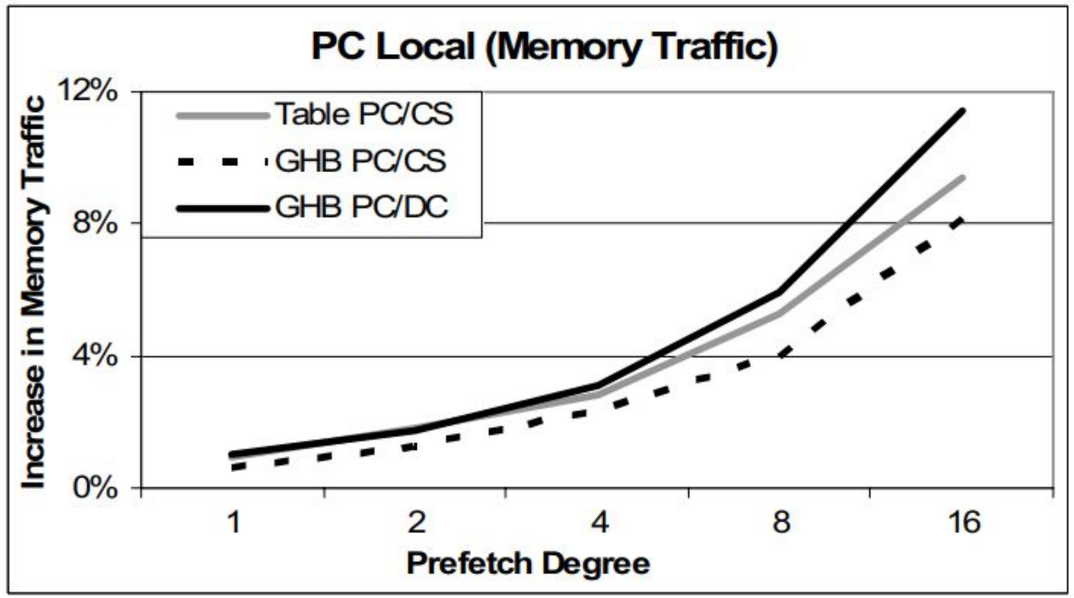


Figure 13: PC Local methods (PC/CS and PC/DC) Increase in Memory Traffic for prefetch degrees 1 through 16.

Conclusion

- GHB Pros
  1. reduce stale data
  2. Preserve complete global history
  3. Less hardware requirements
- GHB Cons

  → Requires multiple table accesses
- Additional Contribution

  → The GHB architecture provides a unified prefetching structure.